Backlogged Bandits: Cost-Effective Learning for Utility Maximization in Queueing Networks

Juaren Steiger† Bin Li‡ Ning Lu†

†Department of Electrical and Computer Engineering, Queen's University, Kingston, Ontario, Canada
‡School of Electrical Engineering and Computer Science, Pennsylvania State University, State College, Pennsylvania, USA
Email: {juaren.steiger, ning.lu}@queensu.ca, binli@psu.edu

IEEE INFOCOM 2024 - IEEE Conference on Computer Communications | 979-8-3503-8350-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/INFOCOM52122.2024.10621295

***Abstract*—Bipartite queueing networks with unknown statistics, where jobs are routed to and queued at servers and yield server-dependent utilities upon completion, model a wide range of problems in communications and related research areas (e.g., call routing in call centers, task assignment in crowdsourcing, job dispatching to cloud servers). The utility maximization problem in bipartite queueing networks with unknown statistics is a bandit learning problem where the delayed semi-bandit feedback depends on the server queueing delay. In this paper, we propose an efficient algorithm that overcomes the technical shortcomings of the state-of-the-art and achieves square root regret, queue length, and feedback delay. Our approach also accommodates additional constraints, such as quality of service, fairness, and budgeted cost constraints, with constant expected peak violation and zero expected violation after a fixed timeslot. Empirically, our algorithm's regret is competitive with the state-of-the-art for some problem instances and outperforms it in others, with much lower delay and constraint violation.**

I. Introduction

A bipartite queueing network consists of a set of N job dispatchers (or job type sources) routing jobs to a set of M servers, where each job yields a utility upon its completion that depends on its dispatcher and the server it was routed to. Additionally, jobs cannot be completed instantaneously after they are dispatched and must therefore be queued at the server. We typically do not know the statistics of the job arrivals, service rates, or utilities a priori, and must therefore learn these statistics during network operation. Bipartite queueing networks model a wide range of real world problems in communications, and operations research in general, including job dispatching to cloud servers, task assignment in crowdsourcing, call routing in call centers, packet scheduling over wireless channels, and assigning patients to wards in a hospital [1]–[3]. For example, consider a tech support helpline where customers with different types of issues are routed and queued to call center workers (servers). Here, some workers may excel at solving some types of issues, which is reflected in the resulting customer service review (utility), and we aim to maximize customer satisfaction, with short waiting times until calls are connected.

The utility maximization problem in a bipartite queueing network with unknown statistics can be characterized as a *constrained combinatorial sleeping bandit* problem [4] with *delayed semi-bandit feedback* [5]. Here, the combinatorial super-arms are the routing decisions, which have the sleeping property due to the random job arrivals. The constraints are the ones necessary for queue stability, i.e. that the arrival rate to each server should be less than the service rate. The feedback delay here is the time a job waits in a server queue and the processing time at the server. Unlike most work on learning under stochastic delayed feedback, which considers i.i.d. over time delays [6]–[9], this has the added challenge that the delays are correlated over time and depend on the previous decisions of the router.

Recently, Fu and Modiano [1] considered the utility maximization problem in bipartite queueing networks and developed a novel joint routing and scheduling algorithm called *Join-the-Shortest-Queue-K* (JSQ-K), that splits each server queue into high and low priority virtual queues according to the spanning tree induced by the optimal extreme point of the fluid problem (4). They extend their algorithm to the case with unknown statistics by combining it with the Confidence Ball algorithm [10] for constrained linear bandits to get the *Confidence Ball JSQ-K* (CB-JSQ-K) algorithm. Their algorithms achieve impressive logarithmic regret and queue length bounds for a class of problem instances. The major drawback is that their guarantees do not hold in many instances. For example, consider the simple 2×3 network with Bernoulli statistics given in Figure 1. The spanning forests induced by the optimal extreme points each contain a tree without a slack server (a server with arrival rate lower than service rate). Therefore, from Proposition 2 and Theorem 1 in [1], the logarithmic regret bounds do not hold.

In many problems, job dispatching is often further constrained. For example, job dispatchers should be afforded *quality of service* in terms of not excessively dropping jobs, workers should be treated *fairly* in a crowdsourcing system by being given a sufficient number of tasks to work on, and users of a cloud computing platform may need to adhere to budgeted server cost constraints. A notable recent work in this direction is due to Liu et al. [2], who introduce the *Pessimistic-Optimistic ONline Dispatch* (POND) algorithm to efficiently route jobs to servers, while balancing the tradeoff between regret and constraint violation. However, they do not consider the queueing aspect of the problem and assume jobs are instantly served after being routed to a server.

In this work, we aim to solve the utility maximization problem in bipartite queueing networks with unknown statistics

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:11:40 UTC from IEEE Xplore. Restrictions apply.

and additional constraints, and overcome the drawbacks of the state-of-the-art (CB-JSQ-K and POND). Before highlighting our contributions in detail, we briefly review the related work.

A. Related Work

Our work is related to prior work on constrained bandit learning and bandit learning with delayed feedback. Li et al. [4] first introduced constrained bandit learning to the communications and learning literature with their long-term fairness constraints. They also developed the first of a style of algorithm that combines UCB estimates with virtual queues for the constraints, which have since been widely used in the areas of IoT [11], caching networks [12], federated learning [13], [14], and mobile crowdsensing [15]. Later, Liu et al. [16] introduced stronger anytime cumulative constraints that were decoupled from the bandit arms. Kernelized bandits with constraints were also considered with communications applications [17], [18]. Recently, Steiger et al. [5] considered constrained bandits with switching costs with wireless networking applications.

Since its introduction by Joulani et al. [6], delayed feedback in bandit learning has also attracted much attention for applications in fog networks [19]–[21], mobile networks [22], [23], caching networks [24] and vehicular networks [25]. Theoretical guarantees beyond bounded delays have also been derived [7], [8], [26]. Some recent work has considered constrained bandit learning with delayed feedback [5], [9] with communications applications.

On the problem of utility maximization in bipartite queueing networks with unknown statistics, the most notable recent work is due to Fu and Modiano [1], as previously discussed. Yang et al. [27] also consider a bipartite queueing network, but they focus on queue stability under unknown service rates instead of utility. This is more similar to the study of "queueing bandits" that focuses on minimizing the queue length regret [28]–[31]. Previously, Liu et al. [2] studied job routing in a bipartite network with additional constraints, but they did not consider the delayed feedback due to queueing.

B. Our Contributions

In this paper, we solve the utility maximization problem in bipartite queueing networks with unknown utilities subject to additional constraints such as QoS, fairness, or budgeted cost constraints. This is a constrained combinatorial sleeping bandit problem with delayed feedback tied to server queues, and therefore the delayed feedback is correlated over time. We are able to guarantee $\widetilde{O}(\sqrt{T})$ [1] regret (Theorem 4) for our algorithm *POLAR* (Pessimistic-Optimistic Learning-Assisted Routing), which is optimal in the instance-independent sense up to logarithmic factors for a routing-only algorithm [2], with $O(\sqrt{T})$ queue lengths (Theorem 1) and feedback delay (Theorem 2), which matches the lower bound for any static policy [1]. For the first time, our results also hold for *all* problem instances under the usual assumption of Slater's condition. Unlike the state-of-the-art (JSQ-K-CB [1] and POND [2]), our algorithm is also agnostic to the time horizon and Slater's constant, whereas POND and JSQ-K-CB use the time horizon T in their tuning parameters. The pessimistic-optimistic algorithm from Liu et al. [16] uses Slater's constant in its tuning parameters, which is unknown if the arrival and services rates are unknown.

We also consider the sample path version of the anytime cumulative constraints [16] and consider the *expected peak violation* for these constraints, which is novel compared to the previously considered violation of the expected anytime cumulative constraints [16] (we also include the zero expected violation after a fixed timeslot result for completeness). We prove that POLAR only suffers $O(1)$ peak violation (Theorem 3). This can be used to obtain further results, such as a high-probability guarantee that the sample path violation never exceeds a certain threshold (Corollary 2).

The main challenge here, compared to existing work, is dealing with delayed feedback. To do so, we extend the moment-generating function (MGF) techniques used to study regret by Steiger et al. [5]. We also employ a novel technique to characterize the delay moments using an MGF version of Wald's identity. With careful choices of tuning parameters, we are able to guarantee our main results and show very low queue length, delay, and peak violation in simulations. A key takeaway from this paper is that delayed feedback algorithm design techniques, e.g. from Joulani at al. [6], that were employed in previous learning algorithms for network utility maximization [32] are not necessary in this setting since controlling the queue lengths also has the effect of controlling the feedback delay.

Our algorithm POLAR is "cost-effective" compared to the state-of-the-art (JSQ-K-CB [1] and POND [2]) in two senses:

(i) It has low computational costs per learning epoch. For example, for Bernoulli arrivals and service (e.g. Figure 1), POLAR runs in $O(NM)$ per learning epoch (see Remark 4 for details). On the other hand, JSQ-K-CB must compute a spanning forest of the network and solve $O(NM)$ linear programs, each with NM decision variables (one for each corner of the confidence ball [10]), leading to $\widetilde{O}((NM)^{1+p})$ time for some $p > 2$ using state-of-the-art linear programming methods [33]. POLAR also has more efficient learning: it exploits the semi-bandit feedback of utilities instead of using full bandit feedback and discarding most observations in a learning epoch, as done in JSQ-K-CB.

(ii) It has a low cost in terms of peak constraint violation. For example, for budgeted server cost constraints, the average maximum cost deficit is constant over time and the expected violation is zero after a fixed timeslot. On the other hand, the theoretical constraint violation guarantees for POND are not as strong and only guarantee constant expected violation.

The rest of the paper is organized as follows. In Section II, we introduce the utility maximization in bipartite queueing network problem with additional constraints. In Section III, we present our POLAR algorithm, and in Section IV, we give

[1]For positive f and g, $f(x) = O(g(x)) \iff \limsup_{x\to\infty} \frac{f(x)}{g(x)} < \infty$, and $f(x) = \widetilde{O}(g(x))$ means $f(x) = O(g(x)\log^k x)$ for some constant k.

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:11:40 UTC from IEEE Xplore. Restrictions apply.

our theoretical guarantees for it. In Section V, we discuss our simulation results, and finally, we conclude the paper in Section VI.

II. Problem Formulation

Consider a bipartite network consisting of two independent sets of nodes: the first, a set of N job dispatchers governed by a central router, and the second, a set of M servers. Let $\boldsymbol{z} \in \{0,1\}^{N\times M}$ be the biadjacency matrix of the network modeling locality constraints, i.e., $z_{nm} = 1$ if there is a link between dispatcher n and server m. The system operates in discrete timeslots $t = 1, 2, \ldots$ At the beginning of each timeslot t, $a_n(t)$ unit-sized jobs arrive at each dispatcher n, and each server m offers $c_m(t)$ units of service. We assume the arrival and service processes are both i.i.d. over time and mutually independent among dispatchers and servers, with $\mathbf{E}[a_n(t)] = \lambda_n$ and $\mathbf{E}[c_m(t)] = \mu_m$. We also assume both processes are bounded by a positive constant $a_n(t), c_m(t) < C$ almost surely. The router then makes a routing decision $\boldsymbol{x}(t) \in \mathcal{X}_{\boldsymbol{a}(t)}$ and routes $x_{nm}(t)$ jobs from each dispatcher n to each server m where

$$\mathcal{X}_{\boldsymbol{a}} \triangleq \left\{ \boldsymbol{x} \in \mathbb{N}^{N\times M} : \begin{array}{ll} x_{nm} \le z_{nm} a_n & \forall\, n \in [N], m \in [M] \\ \sum_{m\in[M]} x_{nm} \le a_n & \forall\, n \in [N] \end{array} \right\} \tag{1}$$

is the set of possible routing decisions when $\boldsymbol{a}(t) = \boldsymbol{a}$. We will sometimes denote the route of a routing policy (π) by using the superscript $\boldsymbol{x}^{(\pi)}(t)$ when it is unclear. All incoming jobs that are not immediately routed to servers are dropped. Each server m maintains a FIFO queue of length $Q_m(t)$ for incoming jobs waiting to be served, which we assume is empty at the beginning of timeslot 1. Then each $Q_m(t)$ has Lindley recursion $Q_m(t+1) = \left[Q_m(t) + \sum_{n=1}^{N} x_{nm}(t) - c_m(t)\right]^+$ where $[y]^+ \triangleq \max\{y, 0\}$.

Let $\mathcal{C}_{nm}(t)$ denote the set of all jobs routed from dispatcher n completed by server m by the end of timeslot t. Then $\nabla\mathcal{C}_{nm}(t) \triangleq \mathcal{C}_{nm}(t) \setminus \mathcal{C}_{nm}(t-1)$ is the set of jobs completed *during* timeslot t, which is revealed to the router at the end of the timeslot (note: $\sum_{n=1}^{N} |\nabla\mathcal{C}_{nm}(t)| = c_m(t)$ where $|\cdot|$ denotes cardinality). Due to the queueing delay of the servers, the completion of each job j routed in timeslot t is delayed by $\delta_j(t)$ timeslots. Therefore, $j \in \nabla\mathcal{C}_{nm}(t + \delta_j(t))$ for a job j routed from dispatcher n to server m in timeslot t. On the other hand, let $\mathcal{O}_{nm}(t)$ denote the set of outstanding (yet to be completed) jobs routed from dispatcher n to server m by the end of timeslot t. Then the total number of jobs routed from dispatcher n to server m by the end of timeslot t can be decomposed as

$$\sum_{\tau=1}^{t} x_{nm}(\tau) = |\mathcal{C}_{nm}(t)| + |\mathcal{O}_{nm}(t)|. \tag{2}$$

A. Utility Maximization

Each job $j \in \nabla\mathcal{C}_{nm}(t)$ yields an unknown utility v_{nm}. However, the router is only able to observe the utility $\hat{v}_{nm}(j) \triangleq v_{nm} + \eta_{nm}(j)$ corrupted by zero-mean noise

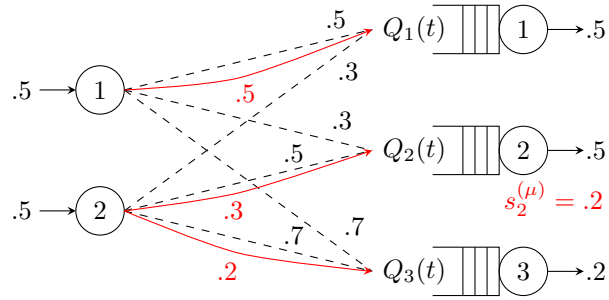


Fig. 1. A 2×3 fully connected queueing network with Bernoulli arrivals $\boldsymbol{\lambda} = (.5, .5)$, Bernoulli offered service $\boldsymbol{\mu} = (.5, .5, .2)$, and utilities (dashed lines) $\boldsymbol{v}_1 = (.5, .3, .7)$, $\boldsymbol{v}_2 = (.3, .5, .7)$. The network has two optimal extreme points (solutions to $\mathscr{F}$), where the first (shown in red arcs) is $r^*_{11} = .5$, $r^*_{22} = .3$, $r^*_{23} = .2$ (server 2 is slack) and the second is $r^*_{11} = .3$, $r^*_{13} = .2$, $r^*_{22} = .5$ (server 1 is slack).

$\eta_{nm}(j)$, which is assumed to be i.i.d. over jobs [2]. Then the primary goal of a router employing routing policy (π) is to make routing decisions to maximize the *total expected utility of completed jobs*

$$\mathrm{Util}^{(\pi)}(T) \triangleq \sum_{n=1}^{N}\sum_{m=1}^{M} v_{nm}\, \mathbf{E}\Big[\big|\mathcal{C}^{(\pi)}_{nm}(T)\big|\Big] \tag{3}$$

up to some time horizon T. Proposition 1 of Fu and Modiano [1] shows that for any dynamic policy (π), including ones that know the unknown problem parameters, $\mathrm{Util}^{(\pi)}(T) \le T\,\mathtt{OPT}(\mathscr{F})$[3] where $\mathtt{OPT}(\mathscr{F})$ is the optimal value of the *fluid* linear program (LP) $\mathscr{F}$:

$$\begin{aligned} \max_{\boldsymbol{r},\boldsymbol{s}^{(\mu)},\boldsymbol{s}^{(\lambda)}} \quad & \sum_{n\in[N]}\sum_{m\in[M]} v_{nm} r_{nm} \\ \text{subject to} \quad & \sum_{m\in[M]} r_{nm} + s_n^{(\lambda)} = \lambda_n && \forall\, n \in [N] \\ & \sum_{n\in[N]} r_{nm} + s_m^{(\mu)} = \mu_m && \forall\, m \in [M] \\ & 0 \le s_n^{(\lambda)}, s_m^{(\mu)} && \forall\, n \in [N],\, m \in [M] \\ & 0 \le r_{nm} \le z_{nm}\lambda_n && \forall\,(n,m) \in [N]\times[M]. \end{aligned} \tag{4}$$

Denote the optimal solution of $\mathscr{F}$ as $\boldsymbol{r}^*$. Each decision variable r_{nm} in $\mathscr{F}$ can be considered the average number of jobs routed from dispatcher n to server m in a timeslot. Each slack variable $s_n^{(\lambda)}$ can be thought of as the average number of jobs dropped from dispatcher n and each slack variable $s_m^{(\mu)}$ can be thought of as the average unused service. Then, we define the *cumulative regret* of a policy (π) up to the horizon T as

$$\mathrm{Reg}^{(\pi)}(T) \triangleq T\,\mathtt{OPT}(\mathscr{F}) - \mathrm{Util}^{(\pi)}(T). \tag{5}$$

[2] Fu and Modiano [1] consider 1-subgaussian noise. We restrict our analysis to the case when $\hat{v}_{nm}(j) \in [0, 1]$, but our algorithm supports both.

[3] They only consider the case where the servers can accommodate all job arrivals and no jobs ever need to be dropped, i.e. $s_n^{(\lambda)} = 0$ for all $n \in [N]$.

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:11:40 UTC from IEEE Xplore. Restrictions apply.

Remark 1. Recall from (2) that $\sum_{\tau=1}^{t} x_{nm}(\tau) = |\mathcal{C}_{nm}(t)| + |\mathcal{O}_{nm}(t)|$. Then, we rewrite

$$\mathrm{Util}^{(\pi)}(T) = \sum_{n=1}^{N}\sum_{m=1}^{M} v_{nm}\,\mathbf{E}\left[\sum_{t=1}^{T} x_{nm}^{(\pi)}(t) - \left|\mathcal{O}_{nm}^{(\pi)}(T)\right|\right] \tag{6}$$

and plugging this into the definition of regret (5) gives the decomposition

$$\begin{aligned}\mathrm{Reg}^{(\pi)}(T) = &\overbrace{\sum_{t=1}^{T}\sum_{n=1}^{N}\sum_{m=1}^{M} v_{nm}\left(r_{nm}^{*} - \mathbf{E}\left[x_{nm}^{(\pi)}(t)\right]\right)}^{\text{cumulative routing regret } \mathrm{XReg}^{(\pi)}(T)} \\ &+ \underbrace{\sum_{n=1}^{N}\sum_{m=1}^{M} v_{nm}\,\mathbf{E}\left[\left|\mathcal{O}_{nm}^{(\pi)}(T)\right|\right]}_{\text{outstanding utility}}.\end{aligned} \tag{7}$$

The cumulative routing regret $\mathrm{XReg}^{(\pi)}(T)$ is the version of regret typically considered in bandit learning problems with or without delayed feedback (see e.g. [5], [6]).

B. Additional Constraints

In addition to the utility maximization goal, the router may need to adhere to W *(sample path) anytime cumulative constraints* [16], where each constraint w is associated with a known linear inequality constraint function g_w that maps

$$\begin{aligned}g_w(\boldsymbol{a}(t), \boldsymbol{c}(t), \boldsymbol{x}(t)) = &\sum_{n\in[N]} \kappa_n^{(w)} a_n(t) + \sum_{m\in[M]} \varrho_m^{(w)} c_m(t) \\ &+ \sum_{n\in[N]}\sum_{m\in[M]} \alpha_{nm}^{(w)} x_{nm}(t) + \beta^{(w)}\end{aligned} \tag{8}$$

where the weights $\kappa_n^{(w)}, \varrho_m^{(w)}, \alpha_{nm}^{(w)} \in [-1, 1]$ for all dispatchers n and servers m, and $\beta^{(w)} \in [-C, C]$. Then each constraint function w is bounded by $|g_w| \leq g_{\max}$ where $g_{\max} \triangleq NC + MC + NMC + C$. We consider the sample path (instead of the expected, as in [16]) version of the anytime cumulative constraints that require that

$$\sum_{\tau=1}^{t} g_w(\boldsymbol{a}(\tau), \boldsymbol{c}(\tau), \boldsymbol{x}(\tau)) \leq 0 \quad \forall\, w \in [W],\, t = 1, 2, \ldots \tag{9}$$

almost surely. In practice, most routing policies will need to violate these constraints to learn the unknown problem parameters. To measure a policy (π)'s adherence to the constraints, we introduce the *expected peak violation* of each constraint w up to the horizon T given by

$$\mathrm{Peak}_w^{(\pi)}(T) \triangleq \mathbf{E}\left[\max_{t\in[T]}\left[\sum_{\tau=1}^{t} g_w(\boldsymbol{a}(\tau), \boldsymbol{c}(\tau), \boldsymbol{x}^{(\pi)}(\tau))\right]^{+}\right]. \tag{10}$$

We augment the fluid LP $\mathscr{F}$ to the fluid LP with additional constraints $\mathscr{W}$ by adding the constraints

$$g_w(\boldsymbol{\lambda}, \boldsymbol{\mu}, \boldsymbol{r}) \leq 0 \qquad \forall\, w \in [W] \tag{11}$$

to (4). Then, following previous work on constrained bandit learning [4], [5], [16], [34], regret in this setting is given by

$$\mathrm{Reg}^{(\pi)}(T) \triangleq T\,\mathtt{OPT}(\mathscr{W}) - \mathrm{Util}^{(\pi)}(T) \tag{12}$$

and Remark 1 still applies where $\boldsymbol{r}^*$ is now the optimal solution to $\mathscr{W}$.

Assumption 1 (Slater's condition)**.** This assumption requires that there exists a feasible solution $\boldsymbol{r}$ to the fluid problem $\mathscr{W}$ such that $s_m^{(\mu)} \geq \gamma$ for each server m and $g_w(\boldsymbol{\lambda}, \boldsymbol{\mu}, \boldsymbol{r}) + \gamma \leq 0$ for each constraint w for some constant $\gamma \in (0, 1]$ (call this the Slater's constant).

To conclude this section, we give three examples of additional constraints in an intuitive form. The values of the weights $\kappa_n^{(w)}, \varrho_m^{(w)}, \alpha_{nm}^{(w)}$ from (8) can be deduced by equating coefficients.

1) Quality of Service Constraints: Consider the case when there are insufficient servers to accommodate the incoming jobs and the router is forced to drop jobs. Assume that each job dispatcher n has a required *delivery ratio* of $\kappa_n \in [0, 1]$, i.e. it requires that at least $\kappa_n a_n(t)$ of its job arrivals are routed in each timeslot t. Then we have $W = N$ constraints where each constraint $w = n$ is characterized by the function $g_n(\boldsymbol{a}(t), \boldsymbol{c}(t), \boldsymbol{x}(t)) = \kappa_n a_n(t) - \sum_{m\in[M]} x_{nm}(t)$. Here, $\mathrm{Peak}_n(T)$ is interpreted as the expected peak excess number of jobs dropped from dispatcher n up to T.

2) Server Fairness Constraints: In many applications, servers should not be left idling indefinitely. For example, workers in a crowdsourcing system should continually be given tasks to work on. This can be formalized as each server m having a *fairness requirement* $\varrho_m \in [0, 1]$ such that the number of jobs routed to it is at least $\varrho_m c_m(t)$ of its offered service in each timeslot t. Then we have $W = M$ constraints where each constraint $w = m$ is characterized by the function $g_m(\boldsymbol{a}(t), \boldsymbol{c}(t), \boldsymbol{x}(t)) = \varrho_m c_m(t) - \sum_{n\in[N]} x_{nm}(t)$. Here, $\mathrm{Peak}_m(T)$ is interpreted as the expected peak unfairness to server m.

3) Budgeted Cost Constraints: Consider the case where each server has a usage cost of $\alpha_m \in [0, 1]$ per job and each dispatcher has a time-average cost budget of $\beta_n \in [0, C]$. Then we have $W = N$ constraints where each constraint $w = n$ is characterized by the function $g_n(\boldsymbol{a}(t), \boldsymbol{c}(t), \boldsymbol{x}(t)) = \sum_{m\in[M]} \alpha_m x_{nm}(t) - \beta_n$. Here, $\mathrm{Peak}_n(T)$ is interpreted as the expected peak cost deficit of dispatcher n.

III. Algorithm Design

We employ the *pessimistic-optimistic* [16] algorithm design strategy to simultaneously maximize utility and adhere to the additional constraints. The *Pessimistic-Optimistic Learning-Assisted Routing (POLAR)* algorithm (Algorithm 1) uses virtual queues to *pessimistically* estimate the size of the server queues and the violation of the additional constraints, and uses UCB estimates to *optimistically* estimate the job utilities. In each timeslot, the algorithm makes the routing decision that maximizes the *pessimistic-optimistic index* that combines these two components (see Remark 3).

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:11:40 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 POLAR

1: Zero-initialize $\widetilde{\boldsymbol{Q}}(1), \widetilde{\boldsymbol{Z}}(1), \boldsymbol{h}(1), \overline{\boldsymbol{v}}(1)$.
2: **for** $t = 1, 2, \ldots$ **do**
3: *Observe* $\boldsymbol{a}(t)$ *job arrivals and offered service* $\boldsymbol{c}(t)$.
4: **for** $(n, m) \in [N] \times [M]$ such that $z_{nm} = 1$ **do**
5: $I_{nm}(t) := \begin{pmatrix} K\sqrt{t}\ \mathtt{UCB}(\overline{v}_{nm}(t), h_{nm}(t), t) \\ -\widetilde{Q}_m(t) - \sum_{w\in[W]} \alpha_{nm}^{(w)} \widetilde{Z}_w(t) \end{pmatrix}$
6: **for** $n \in [N]$ **do**
7: $m^{(n)}(t) := \operatorname{argmax}_{m\in[M]\,:\,z_{nm}=1} I_{nm}(t)$
8: $x_{n\,m^{(n)}(t)}(t) := a_n(t)\,\mathbb{1}\{I_{n\,m^{(n)}(t)}(t) \geq 0\}$
9: **for** $m \neq m^{(n)}(t) \in [M]$ **do** $x_{nm}(t) := 0$
10: Route job arrivals to servers according to $\boldsymbol{x}(t)$.
11: *Observe noisy utilities of completed jobs* $\nabla\mathcal{C}_{nm}(t)$ *from each server* m *for each dispatcher* n.
12: **for** $m \in [M]$ **do** Update $\widetilde{Q}_m(t)$ using (15).
13: **for** $w \in [W]$ **do** Update $\widetilde{Z}_w(t)$ using (16).
14: **for** $(n, m) \in [N] \times [M]$ **do**
15: Update $h_{nm}(t)$ using (14) and $\overline{v}_{nm}(t)$ using (13).

The algorithm takes as input a tuning parameter K and a function UCB that maps the empirical mean utility $\overline{v}_{nm}(t)$, the number of utility observations $h_{nm}(t)$, and the current timeslot t to a UCB estimate $U_{nm}(t) \triangleq \mathtt{UCB}(\overline{v}_{nm}(t), h_{nm}(t), t)$ of v_{nm}. The following remark gives two such functions for different assumptions on the noise distribution.

Remark 2 (UCB functions)**.** Let $Y_1, Y_2, \ldots$ be i.i.d. random variables, each with mean y, and let $\overline{y} = \frac{1}{h}\sum_{i=1}^{h} Y_i$ be the empirical mean of the first h random variables. If each $|Y_i - y|$ is 1-subgaussian and $h > 0$, then the function $\mathtt{UCB}^{(1\text{-sg})}(\overline{y}, h, t) \triangleq \overline{y} + \sqrt{\frac{2}{h}\log\bigl(1 + t\log^2 t\bigr)}$ upper bounds y with high probability when t is large [35]. On the other hand, if each $Y_i \in [0, 1]$, then the function $\mathtt{UCB}^{(0\text{-}1)}(\overline{y}, h, t) \triangleq \min\Bigl\{1,\, \overline{y} + \sqrt{\frac{3}{2h}\log t}\Bigr\}$ upper bounds y with high probability when t is large [4]. With $h = 0$ samples, we initialize $\mathtt{UCB}^{(1\text{-sg})}(\overline{y}, 0, t) \triangleq \infty$ and $\mathtt{UCB}^{(0\text{-}1)}(\overline{y}, 0, t) \triangleq 1$.

For each dispatcher n and server m, the empirical mean utility is initialized to $\overline{v}_{nm}(1) = 0$ and updated as

$$\overline{v}_{nm}(t+1) = \frac{h_{nm}(t)\overline{v}_{nm}(t) + \sum_{j\in\nabla\mathcal{C}_{nm}(t)} \hat{v}_{nm}(j)}{\max\{1, h_{nm}(t+1)\}} \tag{13}$$

and the number of utility observations is initialized to $h_{nm}(1) = 0$ and updated as

$$h_{nm}(t+1) = h_{nm}(t) + |\nabla\mathcal{C}_{nm}(t)|. \tag{14}$$

Then the optimistic components of the algorithm are the UCB estimates $U_{nm}(t) = \mathtt{UCB}(\overline{v}_{nm}(t), h_{nm}(t), t)$, which are weighted by $K\sqrt{t}$.

The pessimistic components are the virtual queues. We use M virtual queues $\widetilde{\boldsymbol{Q}}(t)$ to estimate the size of the server queues, each initialized to $\widetilde{Q}_m(1) = 0$ and updated as

$$\widetilde{Q}_m(t+1) = \left[\widetilde{Q}_m(t) + \sum_{n=1}^{N} x_{nm}(t) - c_m(t) + \varepsilon_t\right]^+. \tag{15}$$

The tuning parameter $\varepsilon_t \triangleq \frac{\mathrm{d}}{\mathrm{d}t}\sqrt{t\log(t+1)}$ makes $\widetilde{Q}_m(t)$ a pessimistic overestimate of $Q_m(t)$ which helps keep the size of the server queue, (and by extension, the feedback delay) small.

We use W virtual queues $\widetilde{\boldsymbol{Z}}(t)$ to estimate the violation of the additional constraints, each initialized to $\widetilde{Z}_w(1) = 0$ and updated as

$$\widetilde{Z}_w(t+1) := \Bigl[\widetilde{Z}_w(t) + g_w(\boldsymbol{a}(t), \boldsymbol{c}(t), \boldsymbol{x}(t)) + \varepsilon_t\Bigr]^+. \tag{16}$$

Here, ε_t works in conjunction with the weight $K\sqrt{t}$ on the UCB estimates to lower the peak violation.

As illustrated in the following remark, the routing decision $\boldsymbol{x}(t)$ in timeslot t is computed by maximizing the *pessimistic-optimistic index* on lines 4-9 of Algorithm 1.

Remark 3 (pessimistic-optimistic index)**.** For convenience, define $q_m^{(\pi)}(t) \triangleq \sum_{n=1}^{N} x_{nm}^{(\pi)}(t) - c_m(t)$ and abuse notation to define $g_w^{(\pi)}(t) \triangleq g_w(\boldsymbol{a}(t), \boldsymbol{c}(t), \boldsymbol{x}^{(\pi)}(t))$ for a policy (π) for all servers m and additional constraints w. The *pessimistic-optimistic index* for the policy in timeslot t is given by

$$\begin{aligned}\mathtt{PO}^{(\pi)}(t) \triangleq\ & K\sqrt{t}\sum_{n\in[N]}\sum_{m\in[M]} x_{nm}^{(\pi)}(t)\,U_{nm}(t) \\ & - \Bigl\langle \widetilde{\boldsymbol{Q}}(t), \boldsymbol{q}^{(\pi)}(t)\Bigr\rangle - \Bigl\langle \widetilde{\boldsymbol{Z}}(t), \boldsymbol{g}^{(\pi)}(t)\Bigr\rangle\end{aligned} \tag{17}$$

where $\langle\,\cdot\,,\,\cdot\,\rangle$ denotes the standard inner product of Euclidean vectors. Now observe from Algorithm 1 that for POLAR,

$$\begin{aligned}\mathtt{PO}(t) = & \overbrace{\sum_{n\in[N]}\sum_{m\in[M]} x_{nm}(t) I_{nm}(t)}^{\text{maximized by POLAR}} \\ & \underbrace{- \sum_{w\in[W]} \widetilde{Z}_w(t)\left(\langle\boldsymbol{\kappa}^{(w)}, \boldsymbol{a}(t)\rangle + \langle\boldsymbol{\varrho}^{(w)}, \boldsymbol{c}(t)\rangle + \beta^{(w)}\right)}_{\text{does not depend on the routing decision } \boldsymbol{x}(t)}.\end{aligned} \tag{18}$$

Therefore, POLAR is a routing policy that maximizes the pessimistic-optimistic index in each timeslot.

Remark 4 (computational complexity of POLAR)**.** For each timeslot t, Algorithm 1 takes $O(NMW)$ time to compute the routing decision $\boldsymbol{x}(t)$. Then, it takes $O(NMW)$ time to update the virtual queues. Finally, Algorithm 1 takes $O(NMC)$ time to update $\overline{\boldsymbol{v}}(t)$ and $\overline{\boldsymbol{h}}(t)$. Then overall, POLAR runs in $O(NM(W \vee C))$ time per timeslot, where $y_1 \vee y_2 \triangleq \max(y_1, y_2)$.

IV. Main Results

In this section, we present our theoretical guarantees for the POLAR algorithm, which are summarized in Table I. For regret, $\widetilde{O}(\sqrt{T})$ is a tight instance-independent bound up to logarithmic factors [2]. $O(1)$ mean peak violation is the best possible. For mean queue length, and correspondingly job feedback delay, $O(\sqrt{T})$ matches the lower bound for any static policy [1] and guarantees *mean-rate stability* [36].[4]

[4]If a constant tuning parameter were used on the UCB estimates, $O(1)$ queue length and *strong stability* may be guaranteed, but at the cost of linear regret [4].

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:11:40 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF RESULTS FOR THE POLAR ALGORITHM

Quantity	Upper Bound	Theorem
Mean Server Queue Length	$O(\sqrt{T})$	Theorem 1
Mean Job Feedback Delay	$O(\sqrt{T})$	Theorem 2
Mean Peak Violation	$O(1)$	Theorem 3
Cumulative Regret	$\widetilde{O}(\sqrt{T})$	Theorem 4

However, our simulation results show much smaller queue length growth for POLAR compared to the state-of-the-art, leading us to believe better bounds may be possible. Due to space limitations, we must omit most of the proofs. For the results in this section, we assume the noisy utilities $\hat{v}_{nm}(j) \in [0,1]$ and the POLAR algorithm uses $\texttt{UCB}^{(0\text{-}1)}$ as its UCB function. When the superscript denoting the policy is omitted, then the POLAR algorithm is being used.

We introduce some preliminary notation. Let $\widetilde{\boldsymbol{QZ}}(t) \triangleq (\widetilde{Q}_1(t), \dots \widetilde{Q}_M(t), \widetilde{Z}_1(t), \dots \widetilde{Z}_W(t))$ denote the vector resulting from the concatenation of $\widetilde{\boldsymbol{Q}}(t)$ and $\widetilde{\boldsymbol{Z}}(t)$. To deal with the tuning parameters $\{\varepsilon_t\}_{t=1,2,\dots}$, we introduce the notation $\overline{\varepsilon}_y \triangleq \inf\{t = 1,2,\dots : \varepsilon_t \leq y\}$ to denote the first round in which the tuning parameter is smaller than the quantity $y > 0$. Define the quadratic Lyapunov function $V(t) \triangleq \frac{1}{2}\sum_{m=1}^{M} \widetilde{Q}_m^2(t) + \frac{1}{2}\sum_{w=1}^{W} \widetilde{Z}_w^2(t)$, which admits the common elementary drift bound (see e.g. [34])

$$\Delta V(t) \leq \varepsilon_t \left\| \widetilde{\boldsymbol{QZ}}(t) \right\|_1 + \left\langle \widetilde{\boldsymbol{Q}}(t), \boldsymbol{q}(t) \right\rangle + \left\langle \widetilde{\boldsymbol{Z}}(t), \boldsymbol{g}(t) \right\rangle + C_V \tag{19}$$

where Δ is the forward difference operator and $C_V \triangleq \frac{1}{2}\left[M(C+1)^2 + W(g_{\max}+1)^2\right]$ is a constant.

One final preliminary step is to derive a class of stationary randomized routing policies, which are used in the proofs of most of our theoretical results. For a feasible solution $\boldsymbol{r}$ to $\mathscr{W}$ with corresponding slack variables $\boldsymbol{s}^{(\lambda)}$, we can derive a *stationary randomized routing policy* $(\boldsymbol{r})$ that routes each incoming job from dispatcher n to server m with probability r_{nm}/λ_n or drops the job with probability $s_n^{(\lambda)}/\lambda_n$. In other words, each $(\boldsymbol{x}_n^{(\boldsymbol{r})}(t) \mid a_n(t) = a_n)$ is a random vector drawn from a Multinomial distribution with a_n trials (since there are a_n job arrivals) and $M+1$ mutually exclusive events (route each job to one of the M servers or drop it). Then for each pair (n,m), it can easily be calculated using the law of total expectation that the policy is expected to route $\mathbf{E}\left[x_{nm}^{(\boldsymbol{r})}(t)\right] = r_{nm}$. We denote the optimal stationary policy by $(*) \triangleq (\boldsymbol{r}^*)$, which has $\sum_{n=1}^{N}\sum_{m=1}^{M} v_{nm}\,\mathbf{E}\left[x_{nm}^{(*)}(t)\right] = \texttt{OPT}(\mathscr{W})$. Also, let (γ) denote the optimal stationary policy under the added requirements that $s_m^{(\mu)} \geq \gamma$ for each server m and $g_w(\boldsymbol{\lambda}, \boldsymbol{\mu}, \boldsymbol{r}) + \gamma \leq 0$ for each constraint w, as in Assumption 1. Then it's easy to check that each $\mathbf{E}\left[q_m^{(\gamma)}(t)\right] \leq -\gamma$ and $\mathbf{E}\left[g_w^{(\gamma)}(t)\right] \leq -\gamma$.

We use the quadratic Lyapunov function along with the Euclidean norm Lyapunov function, and the stationary policy (γ), to derive the following bound on the MGF of the virtual queues, given in Lemma 1. This bound consequently bounds the moments of the server queues in Theorem 1.

Lemma 1. *Under the POLAR algorithm,*

$$\mathbf{E}\left[e^{\zeta \left\| \widetilde{\boldsymbol{QZ}}(t) \right\|_2}\right] \leq e^{\zeta\,(\nu + \nu t_0 + \varphi_t)} \left(1 + \frac{2}{\zeta_{\max}\rho}\right) \tag{20}$$

for all $t \in \mathbb{Z}_+$ and $\zeta \in [0, \zeta_{\max}]$ where $t_0 = \overline{\varepsilon}_{\gamma/3}$, $\rho = \gamma/3$, $\varphi_t = \frac{1}{\rho}\left(C_V + KNMC\sqrt{t}\right)$, $\nu = \sqrt{M+W}(C \vee g_{\max} + 1)$ and $\zeta_{\max} \triangleq \frac{\rho}{\nu^2 + \nu\rho/3}$.

Proof Sketch. Given a $\mathcal{Y}$-valued random process $\{Y(t)\}_{t\in\mathbb{Z}_+}$, a function $f : \mathcal{Y} \to \mathbb{R}$, an increasing positive real-valued sequence $\{\varphi_t\}_{t\in\mathbb{Z}_+}$ and constants $0 < \rho \leq \nu$, if the conditions

$$\begin{aligned} &(i) \quad \mathbb{1}\{f(Y(t)) \geq \varphi_t\}\,\mathbf{E}[\Delta f(Y(t)) \mid Y(t)] \\ &\qquad\quad \leq -\rho \mathbb{1}\{f(Y(t)) \geq \varphi_t\} && \forall\, t \geq t_0, \\ &(ii) \quad \Delta f(Y(t)) \leq \nu && \forall\, t \in \mathbb{Z}_+ \end{aligned}$$

both hold almost surely, then

$$\mathbf{E}\left[e^{\zeta\, f(Y(t))}\right] \leq e^{\zeta\,(\nu + \nu t_0 + \varphi_t)} \left(1 + \frac{2}{\zeta_{\max}\rho}\right) \tag{21}$$

for all $t \in \mathbb{Z}_+$ and $\zeta \in [0, \zeta_{\max}]$ where $\zeta_{\max} \triangleq \frac{\rho}{\nu^2 + \nu\rho/3}$. This statement can be derived by following the proof of Lemma 11 in [16]. Then it suffices to derive the necessary drift conditions for the process $Y(t) = \widetilde{\boldsymbol{QZ}}(t)$ transformed by the function $f(y) = \|y\|_2$. We omit the full proof due to space limitations, but it is similar to previous work [4], [5], [16]. □

An immediate consequence of Lemma 1 are the following queue length bounds given in Theorem 1.

Theorem 1. *Under POLAR, the moments of the length of the longest queue $\|\boldsymbol{Q}(\tau)\|_\infty$ are bounded by*
(i) $\mathbf{E}\left[e^{\zeta\|\boldsymbol{Q}(\tau)\|_\infty}\right] \leq C_1 e^{C_2\zeta\sqrt{t}}$ *for all* $\zeta \in [0, \zeta_{\max}]$,
(ii) $\mathbf{E}[\|\boldsymbol{Q}(\tau)\|_\infty] \leq \frac{1}{\zeta_{\max}}\log C_1 + C_2\sqrt{t}$
for all $\tau \leq t$ where $C_1 \triangleq e^{\zeta_{\max}(1+\nu+\nu t_0 + C_V/\rho)}\left(1 + \frac{2}{\zeta_{\max}\rho}\right)$ and $C_2 \triangleq \frac{KNMC}{\rho}$.

Proof. It can be easily shown by induction that $Q_m(t) \leq \widetilde{Q}_m(t) + \varepsilon_t$ and $\varepsilon_t \leq 1$. Then $\|\boldsymbol{Q}(t)\|_\infty \leq \left\|\widetilde{\boldsymbol{QZ}}(t)\right\|_2 + 1$, and Lemma 1 gives part (i):

$$\mathbf{E}\left[e^{\zeta\|\boldsymbol{Q}(t)\|_\infty}\right] \leq e^{\zeta\,(1+\nu+\nu t_0+\varphi_t)}\left(1 + \frac{2}{\zeta_{\max}\rho}\right) \leq C_1 e^{C_2\zeta\sqrt{t}}. \tag{22}$$

Part (ii) is obtained by applying Jensen's inequality. These bounds also hold for all $\tau \leq t$ since the right hand sides are increasing functions of t. □

From Theorem 1 and using a novel technique applying Wald's identities, we bound the delay MGF in Theorem 2.

Theorem 2. *Under POLAR, the feedback delay $\delta_j(\tau)$ for any job j routed in timeslot $\tau \leq t$ has mean*

$$\mathbf{E}[\delta_j(\tau)] \leq \frac{\mathbf{E}[\|\boldsymbol{Q}(t)\|_\infty] + NC}{p_{\min}} \leq \frac{\log C_1}{\zeta_{\max} p_{\min}} + \frac{C_2\sqrt{t} + NC}{p_{\min}}, \tag{23}$$

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:11:40 UTC from IEEE Xplore. Restrictions apply.

where $p_{\min} \triangleq \min_{m\in[M]} P(c_m(t) > 0)$*, and MGF*

$$\mathbf{E}\Big[e^{\theta\delta_j(\tau)}\Big] \le e^{\zeta NC}\,\mathbf{E}\Big[e^{\zeta\|\boldsymbol{Q}(t)\|_\infty}\Big] \le e^{\zeta NC} C_1 e^{C_2\zeta\sqrt{t}} \tag{24}$$

for $\theta = -\log\big(p_{\min}e^{-\zeta} + 1 - p_{\min}\big)$ *and* $\zeta \in (0, \zeta_{\max}]$.

Proof Sketch. The main idea is to bound the delay by a sum of $\|\boldsymbol{Q}(t)\|_\infty + NC$ i.i.d. Geometric random variables representing the number of timeslots between times when the offered service is positive. We then apply Wald's identity for the mean and the lesser known MGF version of Wald's identity for the MGF. Then Theorem 1 gives the further bounds for the mean and MGF. The full proof is omitted due to space limitations. □

Corollary 1. *Given a time horizon* T *large enough that*

$$\sqrt{\log T/T} \le -\log\big(p_{\min}e^{-\zeta_{\max}} + 1 - p_{\min}\big), \tag{25}$$

for any job j *routed in a timeslot* $t \le T$*,*

$$\mathbf{E}\left[e^{\left(\sqrt{\log T/T}\right)\delta_j(t)}\right] \le e^{\zeta_{\max}NC} C_1 e^{C_3\sqrt{\log T}} \tag{26}$$

under POLAR [5]*, where*

$$C_3 \triangleq \frac{C_2\zeta_{\max}}{-\log(p_{\min}e^{-\zeta_{\max}} + 1 - p_{\min})}. \tag{27}$$

Proof. Take $\zeta = \log p_{\min} - \log\Big(e^{-\sqrt{\log T/T}} - 1 + p_{\min}\Big)$. Since $\sqrt{\log T/T} \le -\log\big(p_{\min}e^{-\zeta_{\max}} + 1 - p_{\min}\big)$, it's easy to check that $\zeta \le \zeta_{\max}$. Then $\sqrt{\log T/T} = -\log\big(p_{\min}e^{-\zeta} + 1 - p_{\min}\big)$. Theorem 2 gives

$$\begin{aligned}
&\mathbf{E}\left[e^{\left(\sqrt{\log T/T}\right)\delta_j(t)}\right] \le e^{\zeta NC} C_1 e^{C_2\zeta\sqrt{t}} \le e^{\zeta NC} C_1 e^{C_2\zeta\sqrt{T}} \\
&= e^{\zeta NC} C_1 \exp\left(\frac{C_2\zeta\sqrt{\log T}}{-\log(p_{\min}e^{-\zeta} + 1 - p_{\min})}\right) \\
&\le e^{\zeta_{\max}NC} C_1 e^{C_3\sqrt{\log T}}.
\end{aligned} \tag{28}$$

□

From Lemma 1, we derive the peak violation bound in Theorem 3. We also include the zero expected violation timeslot (see [16]) for completeness.

Theorem 3. *Under POLAR,*

(i) *the expected peak violation of each additional constraint* w *is bounded by* $\mathrm{Peak}_w(T) \le C_P$ *where* $t' = \left\lceil e^{(1+C_2)^2}\right\rceil$ *and*

$$C_P \triangleq \min_{\zeta\in(0,\zeta_{\max}]} \frac{1}{\zeta}\log\left(1 + C_1\left[\sum_{t=1}^{t'-1}\frac{e^{\zeta C_2\sqrt{t+1}}}{e^{\zeta\sum_{\tau=1}^t \varepsilon_\tau}} + 1 + \frac{2}{\zeta^2}\right]\right). \tag{29}$$

(ii) $\mathbf{E}\Big[\sum_{\tau=1}^t g_w(\tau)\Big] \le 0$ *for* $t \ge t' \vee \left(\frac{1}{\zeta_{\max}}\log C_1\right)^2$*, i.e. we have zero expected violation after a fixed timeslot.*

[5]Note that $e^{C_3\sqrt{\log T}} = O(\sqrt{T})$ (check $T \ge \exp\big(4{C_3}^2\big)$).

Proof Sketch. From iteratively expanding the update equation (16) and applying Lemma 1, we have

$$\mathbf{E}\Big[e^{\zeta\sum_{\tau=1}^t g_w(\tau)}\Big] \le C_1 e^{\zeta[C_2\sqrt{t+1} - \sum_{\tau=1}^t \varepsilon_\tau]}. \tag{30}$$

Then, by bounding the sum $\sum_{\tau=1}^t \varepsilon_\tau$ by an integral, it can be show that $C_2\sqrt{t+1} - \sum_{\tau=1}^t \varepsilon_\tau \le -\sqrt{t}$ for $t \ge t'$. Then we sum over $t = 1, \ldots, T$, bound the sum for $t \ge t'$ by the infinite series $\sum_{t=t'}^\infty e^{-\zeta\sqrt{t}} \le 1 + 2/\zeta^2$, apply the LogSumExp inequality, and use Jensen's inequality to get the final bound (i). The result (ii) proceeds easily by applying Jensen's inequality to (30). We omit the full proof due to space limitations. □

Corollary 2. *For* T *arbitrarily large and any* $y > 1$*,* $P\Big(\sum_{\tau=1}^t g_w(\tau) \le C_P^y\ \forall\, t \le T\Big) \ge 1 - C_P^{1-y}$ *for all* w*.*

Proof. This follows from Theorem 3 part (i) by Markov's inequality. □

Using Corollary 1, Lemma 2 shows that the event that all delays are bounded by $2\sqrt{T\log T}$ has high probability and uses this result to bound the outstanding utility for large enough T.

Lemma 2. *Define the event*

$$\mathcal{D} \triangleq \Big\{\delta_j(t) \le 2\sqrt{T\log T} \quad \forall\, j \text{ routed in timeslots } t \le T\Big\}. \tag{31}$$

Given that T *is large enough that* $\sqrt{\log T/T} \le -\log\big(p_{\min}e^{-\zeta_{\max}} + 1 - p_{\min}\big)$*, under the POLAR algorithm,*

(i) $P(\mathcal{D}) \ge 1 - \Big(NCe^{\zeta_{\max}NC}C_1e^{C_3\sqrt{\log T}}\Big)/T$*,*

(ii) the expected outstanding utility can be bounded by

$$\begin{aligned}
&\sum_{n=1}^N\sum_{m=1}^M v_{nm}\,\mathbf{E}[|\mathcal{O}_{nm}(T)|] \\
&\le 2NMC\sqrt{T\log T} + N^2M^2C^2e^{\zeta_{\max}NC}C_1e^{C_3\sqrt{\log T}}.
\end{aligned} \tag{32}$$

Proof Sketch. Part (i) follows by the union bound and the Chernoff bound using Corollary 1. Part (ii) uses part (i) to separately bound the outstanding reward in the events $\mathcal{D}$ and $\mathcal{D}^{\mathsf{C}}$ by applying the law of total expectation. We omit the full proof due to space limitations. □

Since Lemma 2 bounds the outstanding utility, it remains to bound the cumulative routing regret $\mathrm{XReg}(T)$ to obtain an overall bound on the regret $\mathrm{Reg}(T)$ (see Remark 1). We do so by comparing POLAR against the optimal stationary policy $(*)$. However, to deal with the tuning parameter ε_t, we will need to derive another stationary policy (ε_t) for each timeslot t as a convex combination of $(*)$ and (γ) with conditional distribution

$$\begin{aligned}
&\mathrm{P}(x_{nm}^{(\varepsilon_t)}(t) = x_{nm} \mid a_n(t) = a_n) \\
&= \left(1 - \frac{\varepsilon_t \wedge \gamma}{\gamma}\right)\mathrm{P}(x_{nm}^{(*)}(t) = x_{nm} \mid a_n(t) = a_n) \\
&\quad + \left(\frac{\varepsilon_t \wedge \gamma}{\gamma}\right)\mathrm{P}(x_{nm}^{(\gamma)}(t) = x_{nm} \mid a_n(t) = a_n).
\end{aligned} \tag{33}$$

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:11:40 UTC from IEEE Xplore. Restrictions apply.

Then *the per-round routing regret* of (ε_t) is given by

$$\mathrm{xreg}^{(\varepsilon_t)}(t) = \left(\frac{\varepsilon_t \wedge \gamma}{\gamma}\right) \mathrm{xreg}^{(\gamma)}(t) = \left(\frac{\varepsilon_t \wedge \gamma}{\gamma}\right) \mathrm{xreg}^{(\gamma)}(1). \tag{34}$$

Next, we use the policy (ε_t) to derive an approximation of the per-round routing regret of POLAR given by

$$\widetilde{\mathrm{xreg}}(t) \triangleq \sum_{n=1}^{N} \sum_{m=1}^{M} v_{nm}\, \mathbf{E}\Big[x_{nm}^{(\varepsilon_t)}(t) - x_{nm}(t)\Big]. \tag{35}$$

Then we have the following decomposition of the cumulative regret of the POLAR algorithm.

$$\begin{aligned}
\mathrm{XReg}(T) &= \textstyle\sum_{t=1}^{\overline{\varepsilon}_\gamma - 1} \mathrm{xreg}(t) + \sum_{t=\overline{\varepsilon}_\gamma}^{T} \left(\mathrm{xreg}^{(\varepsilon_t)}(t) + \widetilde{\mathrm{xreg}}(t)\right) \\
&= \textstyle\sum_{t=1}^{\overline{\varepsilon}_\gamma - 1} \mathrm{xreg}(t) + \sum_{t=\overline{\varepsilon}_\gamma}^{T} \left(\left(\frac{\varepsilon_t \wedge \gamma}{\gamma}\right) \mathrm{xreg}^{(\gamma)}(1) + \widetilde{\mathrm{xreg}}(t)\right) \\
&\le \tfrac{\mathtt{OPT}(\mathscr{W})}{\gamma} \textstyle\sum_{t=1}^{T} \varepsilon_t + \sum_{t=\overline{\varepsilon}_\gamma}^{T} \widetilde{\mathrm{xreg}}(t) \\
&\le \tfrac{2\,\mathtt{OPT}(\mathscr{W})}{\gamma} \sqrt{T \log(T+1)} + \textstyle\sum_{t=\overline{\varepsilon}_\gamma}^{T} \widetilde{\mathrm{xreg}}(t)
\end{aligned} \tag{36}$$

where the last inequality bounds $\sum_{t=1}^{T} \varepsilon_t$ by an integral.

The following preliminary bound on the approximate routing regret does not depend on the delayed feedback.

Lemma 3. *The approximate routing regret of POLAR can be bounded by*

$$\begin{aligned}
\sum_{t=\overline{\varepsilon}_\gamma}^{T} \widetilde{\mathrm{xreg}}(t) \le{}& \frac{\mathbf{E}[V(\overline{\varepsilon}_\gamma)]}{K\sqrt{\overline{\varepsilon}_\gamma}} + \frac{2C_V}{K}\sqrt{T} + NMC^2\frac{\pi^2}{6} \\
&+ \sum_{t=\overline{\varepsilon}_\gamma}^{T} \sum_{n=1}^{N} \sum_{m=1}^{M} \mathbf{E}[x_{nm}(t)\,(U_{nm}(t) - v_{nm})].
\end{aligned} \tag{37}$$

Proof Sketch. We omit the proof due to space limitations, but it is similar to previous work using the drift-plus-regret technique with an additional ε_t parameter (see e.g. [16]). □

It remains to bound the part of the routing regret that depends on delayed feedback, which we do in the following lemma.

Lemma 4. *Under the POLAR algorithm, given a time horizon T large enough that $\sqrt{\log T/T} \le -\log\big(p_{\min} e^{-\zeta_{\max}} + 1 - p_{\min}\big)$,*

$$\begin{aligned}
&\sum_{t=\overline{\varepsilon}_\gamma}^{T} \sum_{n=1}^{N} \sum_{m=1}^{M} \mathbf{E}[x_{nm}(t)\,(U_{nm}(t) - v_{nm})] \\
&\le (CN)^2 M e^{\zeta_{\max} NC} C_1 e^{C_3\sqrt{\log T}} \\
&\quad + \frac{NM(C\pi)^2}{6} + NMC\sqrt{60T\log T}.
\end{aligned} \tag{38}$$

Proof Sketch. We generalize the MGF techniques introduced in the regret analysis by Steiger et al. [5], using the results from Lemma 2. We omit the proof due to space limitations. □

We are now ready to give the final regret bound.

Theorem 4. *The cumulative regret of the POLAR algorithm is bounded by*

$$\begin{aligned}
\mathrm{Reg}(T) \le{}& \frac{2NMC}{\gamma}\sqrt{65T\log(T+1)} + \frac{\mathbf{E}[V(\overline{\varepsilon}_\gamma)]}{K\sqrt{\overline{\varepsilon}_\gamma}} + \frac{2C_V\sqrt{T}}{K} \\
&+ \frac{NM(C\pi)^2}{3} + 2(CN)^2 M e^{\zeta_{\max} NC} C_1 e^{C_3\sqrt{\log T}}
\end{aligned} \tag{39}$$

if T is large enough that $\sqrt{\log T/T} \le C_4$ where $C_4 \triangleq -\log\big(p_{\min} e^{-\zeta_{\max}} + 1 - p_{\min}\big)$. Otherwise,

$$\mathrm{Reg}(T) \le NMC\, {C_4}^{-2} \log T. \tag{40}$$

Proof. The first case when T is large enough follows from (7), (36), Lemma 3, Lemma 4, and Lemma 2. The second case follows by rearranging and noting that we always have $\mathrm{Reg}(T) \le NMCT$. □

V. Empirical Results

In this section, we corroborate our theoretical results for POLAR by comparing it against 3 algorithms from the literature in three different settings. We implement our simulations in the Julia [37] programming language and open source our code, so that the reader may replicate the results. Experiments are each repeated 500 times and the empirical mean of each performance metric is plotted along with a ribbon showing its 95% confidence interval. As in [1], we fix the time horizon to $T = 2 \times 10^5$ for all experiments. We set the tuning parameter $K = 2$ for POLAR.

We compare against 2 control (non-learning) policies that are given full knowledge of all unknown problem parameters. The first is the JSQ-K policy with $K = 20 \log T$ due to Fu and Modiano [1]. Keep in mind that this is a joint routing and scheduling policy, whereas POLAR only focuses on routing. The second is the optimal static policy $(*)$.

We also compare against the learning-assisted routing policy POND due to Liu et al. [2] modified for delayed feedback. The primary differences between POND and POLAR are the choice of UCB function and the tuning parameters. POND uses the UCB function $\overline{v}_{nm}(t) + \sqrt{\log T/h_{nm}(t)}$, and therefore requires knowledge of the time horizon T. POND uses the tuning parameter $2\sqrt{T}$ on the UCB term and the tuning parameter $\frac{1}{2}/\sqrt{T}$ in the virtual queues. [6]

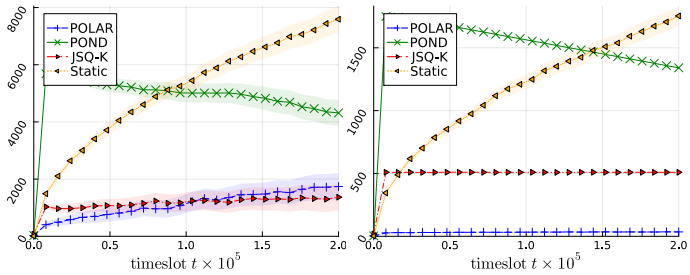
A. Uniform Arrivals and Service and Subgaussian Utilities

This setting uses the exact same problem parameters as Fu and Modiano [1], with $N = 5$ dispatchers, $M = 4$ servers, uniform arrivals and service, and utilities with Gaussian$(0, 1)$ noise. The results are shown in Figure 2.

The results for JSQ-K appear to agree with those found by Fu and Modiano [1] for regret and queue length. Although slightly worse, the regret of POLAR is competitive with that

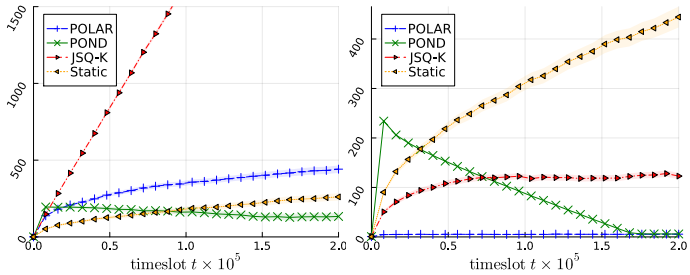
[6] We were unable to get the learning algorithm CB-JSQ-K from Fu and Modiano [1] to converge the way it is written in the paper, but they report approximately 2× worse regret than JSQ-K and comparable queue length over 10 simulation runs.

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:11:40 UTC from IEEE Xplore. Restrictions apply.



(a) $\mathrm{Reg}(t)$ (b) $\|\boldsymbol{Q}(t)\|_1$

Fig. 2. Simulation results in the setting from Fu and Modiano [1].



(a) $\mathrm{Reg}(t)$ (b) $\|\boldsymbol{Q}(t)\|_1$

Fig. 3. Simulation results in the setting from Figure 1.

of JSQ-K. However, POLAR drastically outperforms JSQ-K in terms of queue length, which is proportional to the feedback delay. The static policy performs worse yet with what appears to be square root regret and queue length growth. POND also performs worse than JSQ-K and POLAR, but is appearing to slowly converge toward their results. Fu and Modiano [1] report ≈ 4000 regret for their learning algorithm JSQ-K-CB, which POLAR significantly outperforms.

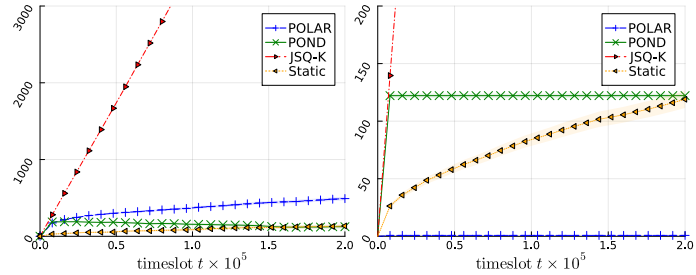
B. Bernoulli Arrivals, Service, and Utilities

This setting uses the problem parameters shown in Figure 1 with $N = 2$ dispatchers, $M = 3$ servers and Bernoulli arrivals, service, and utilities. The results are shown in Figure 3.

In this setting, POND and the static policy appear to perform best for regret, with POLAR behind these two. JSQ-K completely fails in this setting and suffers linear regret. As previously mentioned in Section I, we suspect this is because the spanning forest induced by the optimal extreme point contains a tree without a slack server. The queue lengths of the static policy and JSQ-K are once again large, and POLAR once again has very small queue lengths. POND has initially large queue lengths, but converges to match POLAR in the long run. In this setting, the trend of POND initially suffering poor performance, which improves over time is clearly visible, compared to the previous setting.

C. Bernoulli Setting with Budgeted Server Cost Constraints

This setting is the same as the previous, but with additional constraints in the form of $W = N$ budgeted server cost constraints (see Section II-B3). Namely, the server costs are



(a) $\mathrm{Reg}(t)$ (b) $\sum_{w=1}^{W} \mathrm{Peak}_w(t)$

Fig. 4. Simulation results in the setting from Figure 1 with cost constraints. We omit the figure showing $\|\boldsymbol{Q}(t)\|_1$ due to space limitations, but the trends are the same as in the previous setting.

$\boldsymbol{\alpha} = (.5, .5, .9)$ and the dispatcher budgets are $\boldsymbol{\beta} = (.3, .3)$. The results are shown in Figure 4. The regret results appear more or less the same as the previous setting, with JSQ-K completely failing. POLAR has very low peak violation, the static policy appears to have square root increasing peak violation, and POND has high peak violation, likely due to the initial poor performance as previously mentioned. JSQ-K unsurprisingly suffers linearly increasing peak violation, as it has no mechanism to deal with the additional constraints.

Overall, POLAR significantly outperforms all the policies for queue length and peak violation. In the subgaussian setting, POLAR is competitive for regret with the JSQ-K policy, which has full information (no learning), and outperforms the other policies. While the other learning policy POND appears to beat POLAR for regret in the Bernoulli settings, POLAR still achieves relatively low $\sqrt{t}$ regret growth.

VI. Conclusion

In this paper, we studied the utility maximization problem in bipartite queueing networks with unknown statistics and additional constraints. We developed the POLAR algorithm and proved that it achieves square root queue lengths, feedback delay, and regret, and constant peak constraint violation. Simulation results show that our algorithm performs well against the state-of-the-art, especially for the queueing delay and peak violation. As also mentioned by Fu and Modiano [1], an important future research direction is to consider the feedback delay as part of the utility. This would be an important step in unifying the queueing regret [28] with the traditional bandit regret and would have real world impact, as timely service is important in many applications. Another important future research direction is to extend the problem to multi-hop networks [32]. Generalizing our technique to bound the statistics of the feedback delay in this setting is non-trivial since the queue departures from intermediate nodes are not i.i.d. The authors have provided public access to their code and/or data at https://gitlab.com/juarensteiger/infocom2024.

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:11:40 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] X. Fu and E. Modiano, "Joint learning and control in stochastic queueing networks with unknown utilities," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 6, no. 3, dec 2022. [Online]. Available: https://doi.org/10.1145/3570619

[2] X. Liu, B. Li, P. Shi, and L. Ying, "Pond: Pessimistic-optimistic online dispatching," 2021.

[3] W. Weng, X. Zhou, and R. Srikant, "Optimal load balancing in bipartite graphs," 2020.

[4] F. Li, J. Liu, and B. Ji, "Combinatorial sleeping bandits with fairness constraints," 2019.

[5] J. Steiger, B. Li, and N. Lu, "Learning from delayed semi-bandit feedback under strong fairness guarantees," in *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*, 2022, pp. 1379–1388.

[6] P. Joulani, A. György, and C. Szepesvári, "Online learning under delayed feedback," 2013.

[7] T. Lancewicki, S. Segal, T. Koren, and Y. Mansour, "Stochastic multi-armed bandits with unrestricted delay distributions," 2021.

[8] A. Sawwan and J. Wu, "A new framework: Short-term and long term returns in stochastic multi-armed bandit," in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, 2023.

[9] Q. Liu and Z. Fang, "Learning to schedule tasks with deadline and throughput constraints," in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, 2023.

[10] V. Dani, T. P. Hayes, and S. M. Kakade, "Stochastic linear optimization under bandit feedback," in *Annual Conference Computational Learning Theory*, 2008. [Online]. Available: https://api.semanticscholar.org/CorpusID:9134969

[11] Q. Leng, S. Wang, X. Huang, Z. Shao, and Y. Yang, "Decentralized multi-agent bandit learning for intelligent internet of things systems," in *2022 IEEE Wireless Communications and Networking Conference (WCNC)*, 2022, pp. 2118–2123.

[12] X. Gao, X. Huang, Y. Tang, Z. Shao, and Y. Yang, "History-aware online cache placement in fog-assisted iot systems: An integration of learning and control," *IEEE Internet of Things Journal*, pp. 1–1, 2021.

[13] T. Huang, W. Lin, W. Wu, L. He, K. Li, and A. Zomaya, "An efficiency-boosting client selection scheme for federated learning with fairness guarantee," *IEEE Transactions on Parallel and Distributed Systems*, p. 1–1, 2020. [Online]. Available: http://dx.doi.org/10.1109/TPDS.2020.3040887

[14] W. Xia, T. Q. S. Quek, K. Guo, W. Wen, H. H. Yang, and H. Zhu, "Multi-armed bandit-based client scheduling for federated learning," *IEEE Transactions on Wireless Communications*, vol. 19, no. 11, p. 7108–7123, Nov 2020. [Online]. Available: http://dx.doi.org/10.1109/TWC.2020.3008091

[15] K. Guo and T. Q. S. Quek, "Dynamic computation offloading in multi-server mec systems: An online learning approach," in *GLOBECOM 2020 - 2020 IEEE Global Communications Conference*, 2020, pp. 1–6.

[16] X. Liu, B. Li, P. Shi, and L. Ying, "An efficient pessimistic-optimistic algorithm for stochastic linear bandits with general constraints," 2021.

[17] X. Zhou and B. Ji, "On kernelized multi-armed bandits with constraints," 2022. [Online]. Available: https://arxiv.org/abs/2203.15589

[18] Y. Deng, X. Zhou, A. Ghosh, A. Gupta, and N. B. Shroff, "Interference constrained beam alignment for time-varying channels via kernelized bandits," 2022. [Online]. Available: https://arxiv.org/abs/2207.00908

[19] M. Yang, H. Zhu, H. Qian, Y. Koucheryavy, K. Samouylov, and H. Wang, "Peer offloading with delayed feedback in fog networks," 2021.

[20] Z. Zhu, T. Liu, Y. Yang, and X. Luo, "Blot: Bandit learning-based offloading of tasks in fog-enabled networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 12, pp. 2636–2649, 2019.

[21] X. Gao, X. Huang, and Z. Shao, "Online task offloading with bandit learning in fog-assisted iot systems," in *2019 IEEE 90th Vehicular Technology Conference (VTC2019-Fall)*, 2019, pp. 1–5.

[22] Y. Zhao, H. Qian, K. Kang, and Y. Jin, "Non-stationary bandit strategy for rate adaptation with delayed feedback," *IEEE Access*, vol. 8, pp. 75 503–75 511, 2020.

[23] Y. Zhou, C. Shen, X. Luo, and M. van der Schaar, "A non-stationary online learning approach to mobility management," in *2018 IEEE International Conference on Communications (ICC)*, 2018, pp. 1–6.

[24] Z. Huang, B. Hu, and J. Pan, "Caching by user preference with delayed feedback for heterogeneous cellular networks," *IEEE Transactions on Wireless Communications*, vol. 20, no. 3, pp. 1655–1667, 2021.

[25] L. Chen and J. Xu, "Task replication for vehicular cloud: Contextual combinatorial bandit with delayed feedback," in *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*, 2019, pp. 748–756.

[26] J. Zimmert and Y. Seldin, "An optimal algorithm for adversarial bandits with arbitrary delays," 2020.

[27] Z. Yang, R. Srikant, and L. Ying, "Learning while scheduling in multi-server systems with unknown statistics: Maxweight with discounted ucb," in *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, F. Ruiz, J. Dy, and J.-W. van de Meent, Eds., vol. 206. PMLR, 25–27 Apr 2023, pp. 4275–4312. [Online]. Available: https://proceedings.mlr.press/v206/yang23d.html

[28] S. Krishnasamy, R. Sen, R. Johari, and S. Shakkottai, "Regret of queueing bandits," in *Proceedings of the 30th International Conference on Neural Information Processing Systems*, ser. NIPS'16. Red Hook, NY, USA: Curran Associates Inc., 2016, p. 1677–1685.

[29] S. Krishnasamy, A. Arapostathis, R. Johari, and S. Shakkottai, "On learning the $c\mu$ rule in single and parallel server networks," 2018.

[30] S. Krishnasamy, R. Sen, R. Johari, and S. Shakkottai, "Learning unknown service rates in queues: A multiarmed bandit approach," *Operations Research*, vol. 69, no. 1, pp. 315–330, 2021. [Online]. Available: https://doi.org/10.1287/opre.2020.1995

[31] T. Stahlbuhk, B. Shrader, and E. Modiano, "Learning algorithms for minimizing queue length regret," 2020.

[32] X. Fu and E. Modiano, "Learning-num: Network utility maximization with unknown utility functions and queueing delay," vol. 30, no. 6, p. 2788–2803, jun 2022. [Online]. Available: https://doi.org/10.1109/TNET.2022.3182890

[33] M. B. Cohen, Y. T. Lee, and Z. Song, "Solving linear programs in the current matrix multiplication time," 2020.

[34] J. Steiger, B. Li, B. Ji, and N. Lu, "Constrained bandit learning with switching costs for wireless networks," in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, 2023.

[35] T. Lattimore and C. Szepesvári, *Bandit Algorithms*. Cambridge University Press, 2020.

[36] M. J. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan and Claypool Publishers, 2010.

[37] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017. [Online]. Available: https://doi.org/10.1137/141000671

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:11:40 UTC from IEEE Xplore. Restrictions apply.